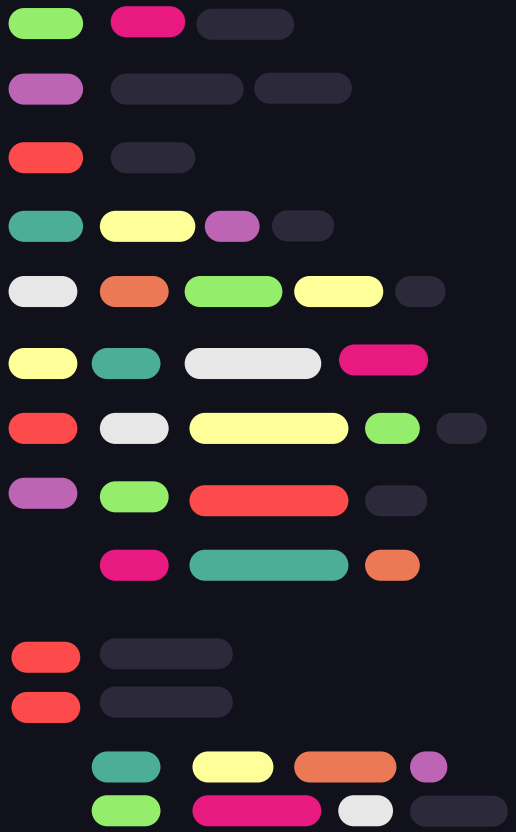


PopOut

MCTS & Árvores de Decisão

Inteligência Artificial – FCUP 2025/26

Bruno Zabot – up202302069
Joseph Campbell – up202408486
Vitor Ferreira – up202205699

01 ..

O jogo PopOut

..

Nossa implementação:

- Jogado no terminal com cores
- Setas para mostrar possíveis jogadas
- Internamente:
  - Jogadores são representados na matriz com 1 para vermelho, -1 para azul e 0 para vazio
  - Movimentos são representados em uma tupla ([coluna], [ação])

Como os outros “modos de jogo” foram implementados:

- As classes que implementam os modos de jogo contra os agentes, herdam os métodos da classe PopOut

```
class PopOutMCTS(PopOut):
```

```
class PopOutDT(PopOut):
```

```
class PopOutMCTS(PopOut):  1 usage
    def game_loop(self, human_player=RED_PLAYER, mcts_iterations=1500):  1 usage
        mcts = MCTS(iterations=mcts_iterations)

        while not self.is_terminal():
            self.print_board()

            if self.turn == human_player:
                move = self.get_player_input(self.is_full_board())
                if move == DRAW_MOVE:
                    self.apply_move(move)
                    self.print_board(show_arrow=False)
                    print("Draw")
                    break
                self.apply_move(move)
            else:
                print("MCTS is thinking...")
                move = mcts.search(self)
                self.apply_move(move)
                print(f"MCTS played: {move}")
                if move == DRAW_MOVE:
                    self.print_board(show_arrow=False)
                    print("Draw")
                    break
            self.print_board(show_arrow=False)

            if self.is_terminal():
                winner = self.check_winner()
                if winner is None:
                    print("Draw")
                else:
                    self.print_winner(winner)
```

02 ..

Monte Carlo Tree Search

MCTS como serviço

- Implementação genérica, para qualquer tipo de jogo, desde que implemente o contrato
- Entrada: estado atual
- Contrato:
  - *clone*
  - *available_moves*
  - *apply_move*
  - *is_terminal*
  - *check_winner*
- Saída: melhor movimento
- Para o PopOut:
  - (coluna, “d”)
  - (coluna, “p”)

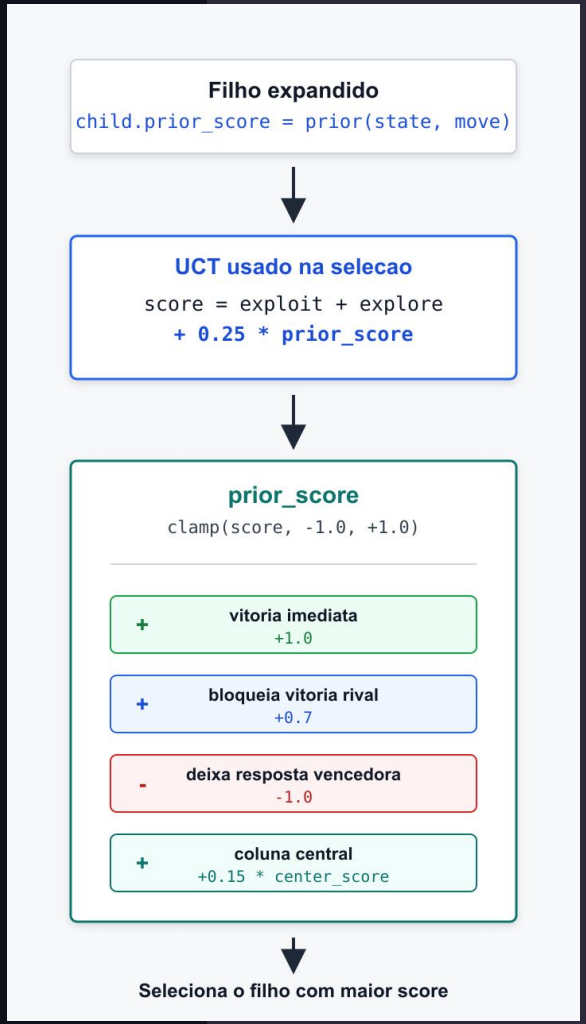
Melhorias táticas

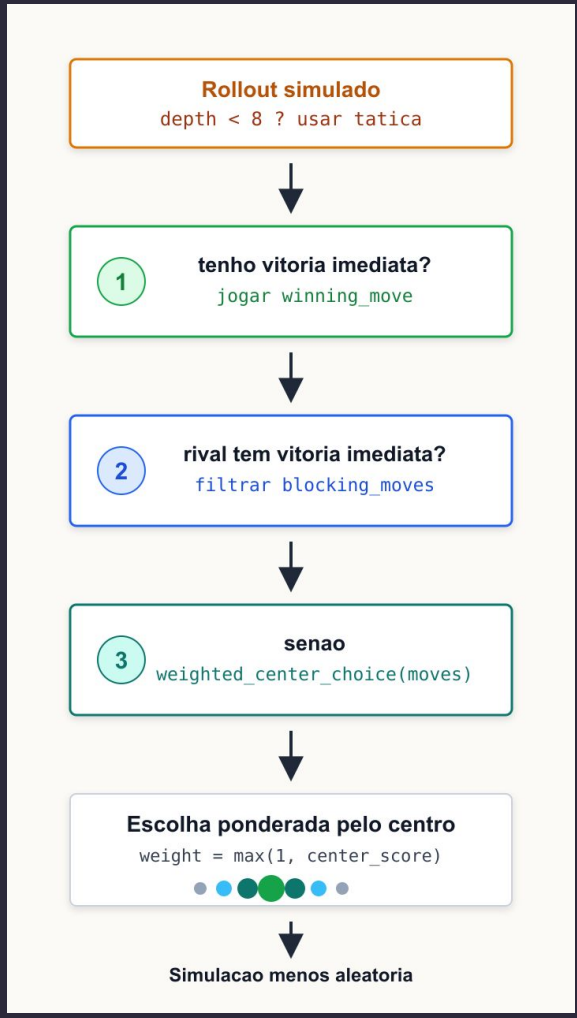
UCT

- Viés heurístico
- Adiciona ou subtrai scores baseado em movimentos

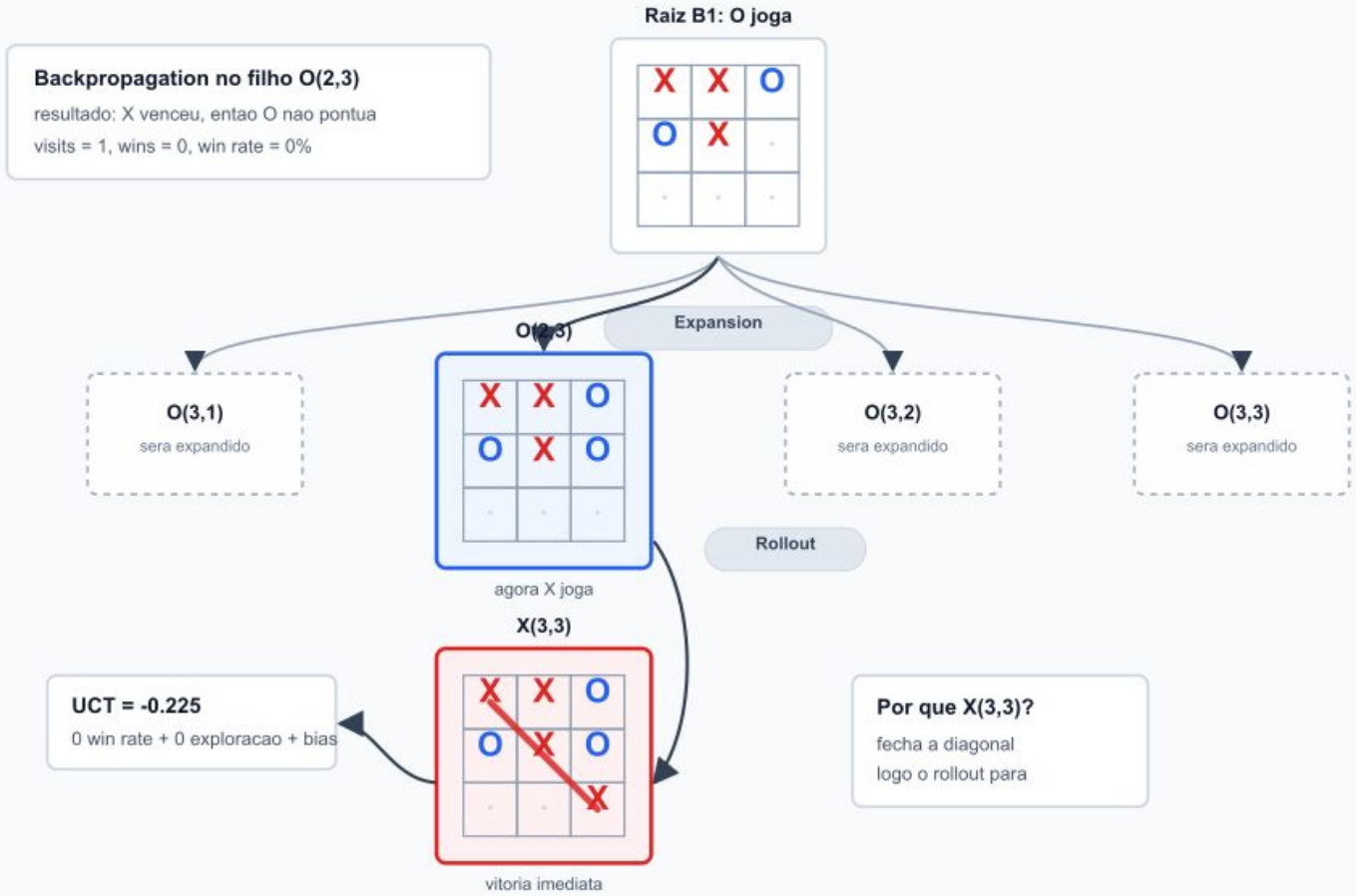
Táticas para o Rollout

- Verifica se tem algum movimento que ganharia o jogo
- Verifica bloquear adversário
- Prefere jogar pelo centro





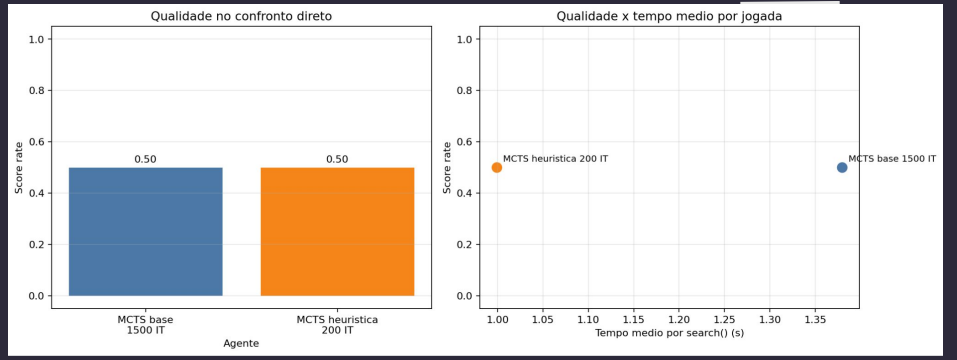
O joga agora. A arvore guarda estados; o rollout so estima o resultado daquele ramo.

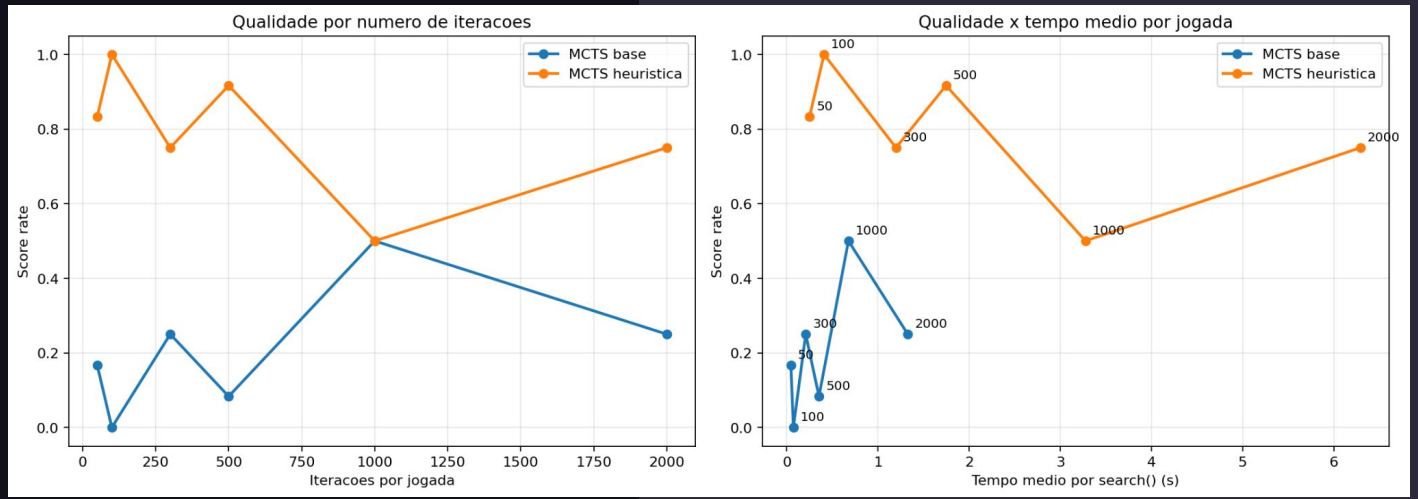


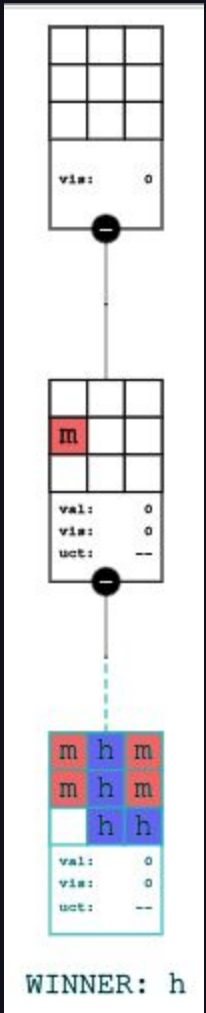
Resultados Melhorias

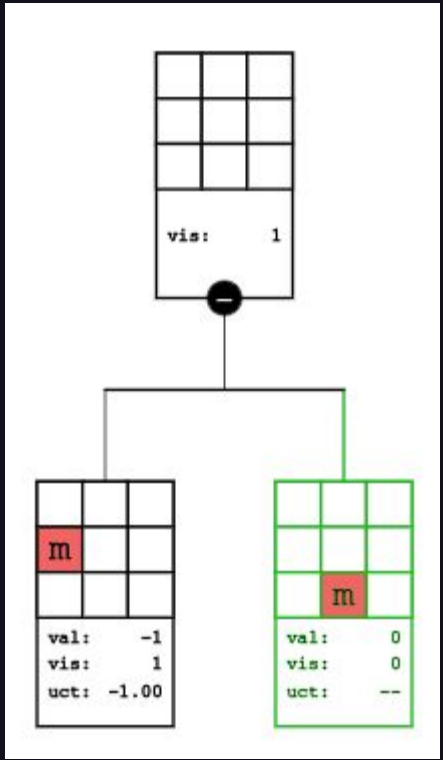
mcts_quality_time_comparison.ipynb

- Compara MCTS base vs heurística
- Qualidade: score rate no confronto direto (percentual de vitória)
- Inclui caso de **tempo** parecido: heurística com menos iterações contra base com mais iterações






vis: 0
m
val: 0
vis: 0
uct: --
m h m
m h m
h h
val: 0
vis: 0
uct: --
WINNER: h


vis: 1
m
val: -1
vis: 1
uct: -1.00
m
val: 0
vis: 0
uct: --

03 ..

Decision Tree

Objectivo: Implementar uma Decision Tree ID3 para classificar o Iris dataset

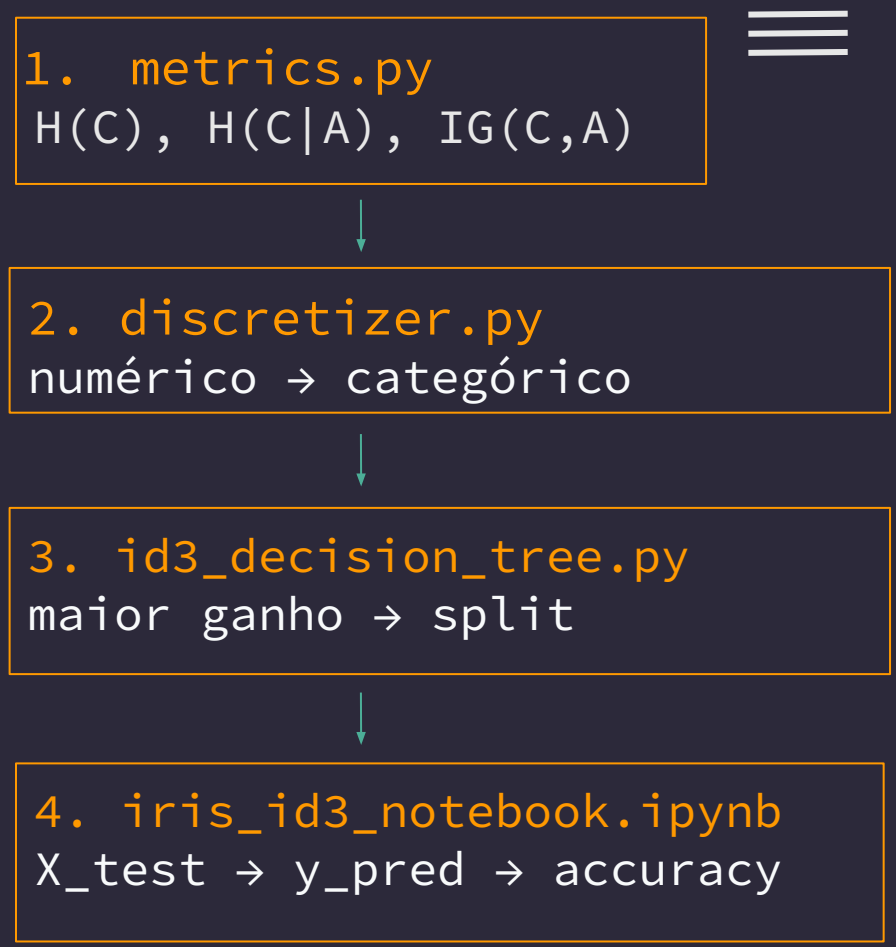
Código Desenvolvido:

1. **Métricas:** entropia e ganho de informação

2. **Discretização:** criar bins para atributos contínuos

3. **ID3:** construção recursiva da árvore

4. **Integração:**treino, predição e accuracy



Pipeline Experimental - Iris

1. Analisar dataset Iris:
150 exemplos, 4 features contínuas, 3 classes

2. Divisão treino/teste
separar dados para treino e teste

3. Aplicar Discretização
converter valores contínuos em bins

4. Treinar ID3 no dataset Iris
escolher splits por ganho de informação

5. Avaliação
medir accuracy no test split

3. Aplicar Discretização:

equal_width

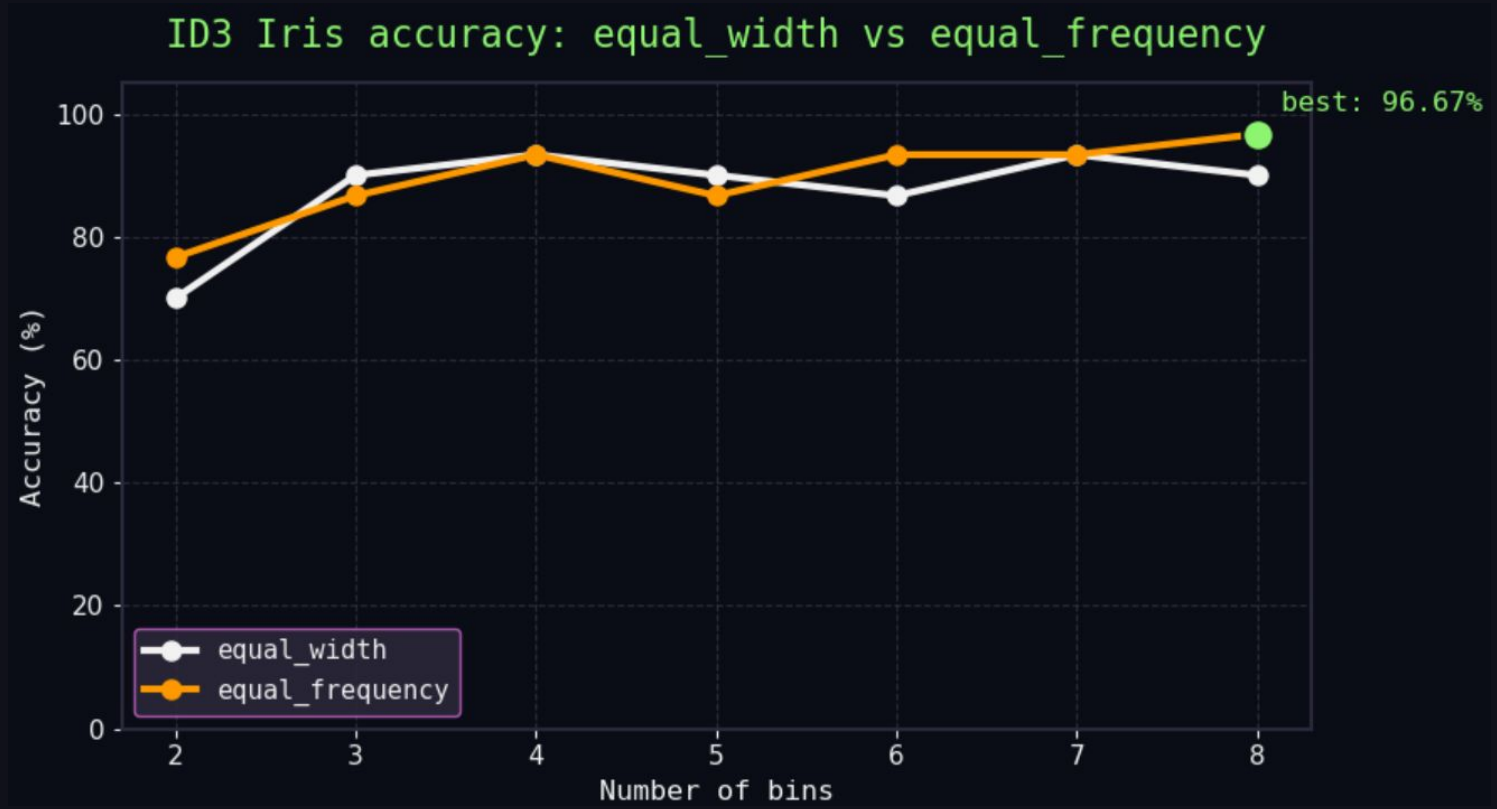
Intervalos com a mesma largura

equal_frequency

bins com n.º semelhante de exemplos

ID3 Iris - Resultados

n_bins	equal_frequency	equal_width
2	76.67% (23/30)	70.00% (21/30)
3	86.67% (26/30)	90.00% (27/30)
4	93.33% (28/30)	93.33% (28/30)
5	86.67% (26/30)	90.00% (27/30)
6	93.33% (28/30)	86.67% (26/30)
7	93.33% (28/30)	93.33% (28/30)
8	96.67% (29/30)	90.00% (27/30)


ID3 Iris accuracy: equal_width vs equal_frequency
best: 96.67%
Accuracy (%)
Number of bins
equal_width
equal_frequency
0
20
40
60
80
100
2
3
4
5
6
7
8

04 ..

Aplicando a Decision Tree ao PopOut

..

Preparação dos datasets:

1. MCTS x Random:
   - 6.000 jogos
   - MCTS com 10.000 iterações
2. MCTS x MCTS:
   - 20 jogos
   - MCTS com 10.000 iterações
   - Mais jogos geraria exemplos iguais
3. Junção dos datasets:
   - Remoção dos exs duplicados
4. Treino do DT:
   - Primeiro modelo DT

Data aggregation para o DT final:

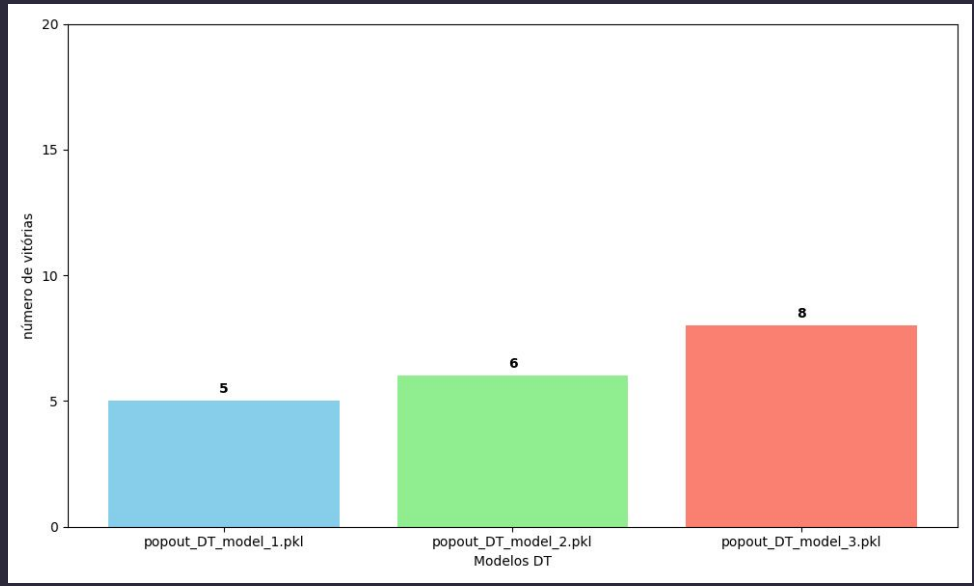
1. MCTS x DT:
   - 20 jogos
   - MCTS com 10.000 iterações
   - Salvando apenas as decisões do MCTS

2. Junção com último dataset e treino de um novo DT

3. MCTS x DT novo:
   - 600 jogos
   - MCTS com 10.000 iterações

4. Junção dos datasets e treino do DT final:
   - Dataset com X exemplos únicos

Vitórias dos modelos em 20 jogos contra um MCTS com 500 iterações:



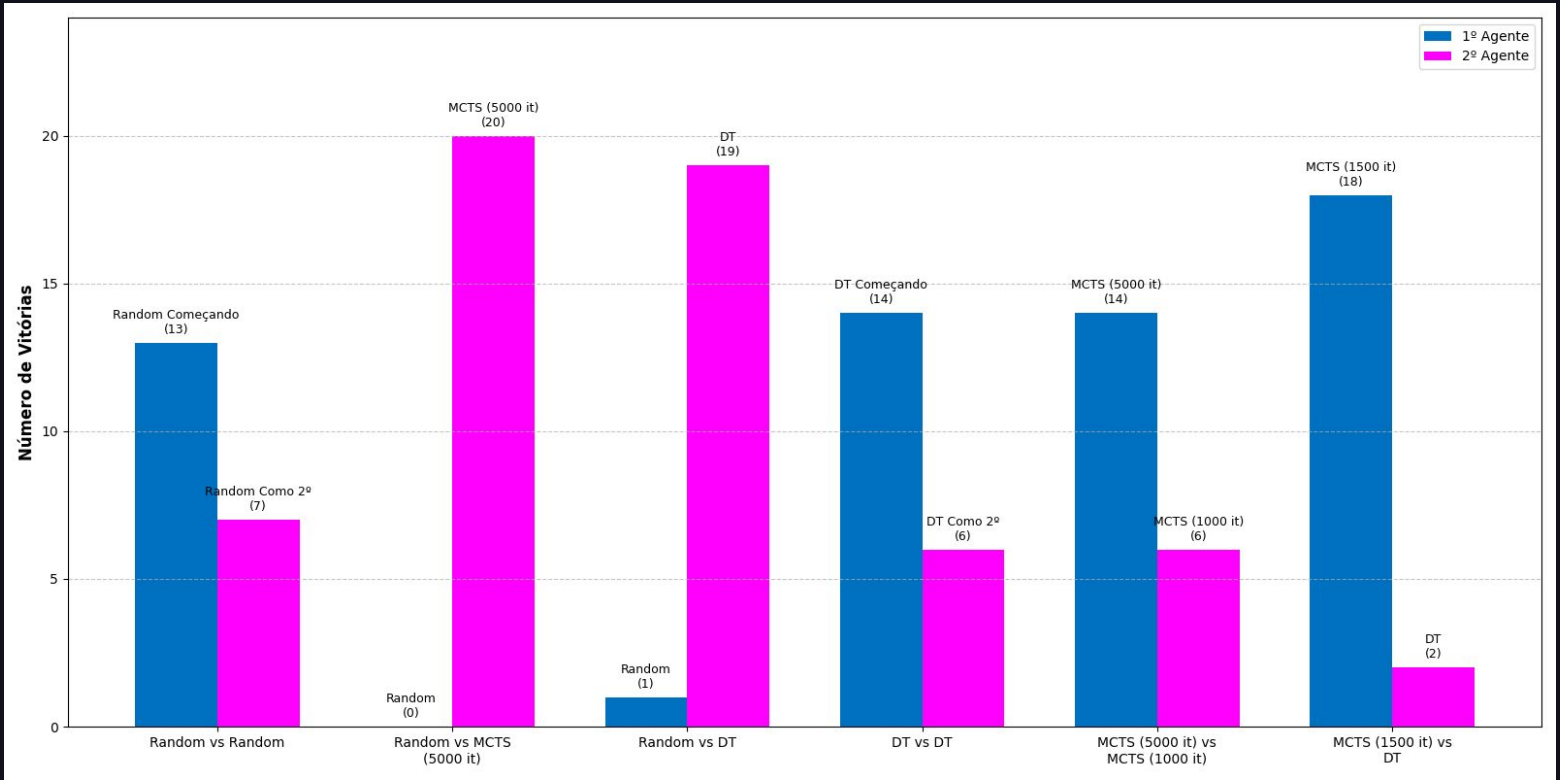
05 ..

Resultados

..

Performance dos agentes

Número de vitórias em 20 partidas



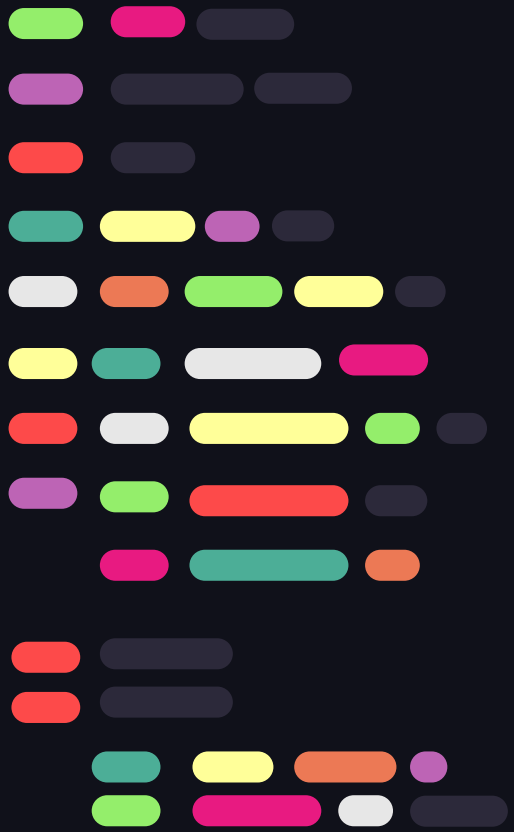
Conclusão

- MCTS domina todos os testes
- Escalabilidade do MCTS
  - Mais iterações = melhor performance
- Limitações do Decision Tree
  - Não consegue jogar tão bem com um dataset de tamanho razoável

Random vs MCTS (5000 it)	
0	20

MCTS (1500 it) vs DT	
18	2

MCTS (5000 it) vs MCTS (1000 it)	
14	6



Fim